Geavanceerde Computerarchitectuur
Session 2

Christoph Sanders & Wout Corijn

2025-02-17

1 Probleem

In dit labo wordt onderzocht hoe de maximale waarde uit een array kan worden bepaald met drie verschillende methoden: een geneste for-lus op de CPU, een GPU-kernel met atomaire operaties en een GPU-kernel met reductie. Daarnaast wordt de afhankelijkheid van de uitvoeringstijd van de invoerlengte geanalyseerd en vergeleken.

2 Methode

2.1 CPU (Nested for-loop)

De maximum waarde in de array wordt gevonden door elke waarde met elke andere waarde te vergelijken. Dit wordt geïmplementeerd met 2 geneste for-loops.

2.2 GPU kernel (atomic operation)

Eerst wordt het nodige geheugen op de GPU gealloceerd, zowel voor de lijst als voor een integer die de maximale waarde zal bevatten. Beide geheugenlocaties worden vervolgens gevuld met data. Er worden evenveel threads aangemaakt als er elementen in de lijst zijn, waarbij elke thread een (max) atomaire operatie uitvoert op zijn overeenkomstige element en de maximale waarde. Het voordeel van deze atomaire operaties is dat de synchronisatie tussen de verschillende blokken wordt gewaarborgd.

2.3 GPU kernel (reduction)

De parallelle reduction-methode vergelijkt in iteraties paren van elementen om de maximale waarde te bepalen, waarbij elk paar door afzonderlijke threads wordt verwerkt. Na elke iteratie zorgt __syncthreads() ervoor dat alle threads binnen hetzelfde blok wachten, zodat de verdere reductie veilig en synchroon verloopt. Bij de initialisatie worden er evenveel threads aangemaakt als de helft van het aantal elementen, en het aantal actieve threads wordt per iteratie gehalveerd totdat één maximum overblijft.

3 Resultaat

Voor elke methode zijn vijf testen uitgevoerd die de de verwerkingstijd van het programma meten van array grootte 2 tot 4096, met als stap veelvouden van 2. Onderstaande grafiek toont de gemiddelde waarden van deze testen.

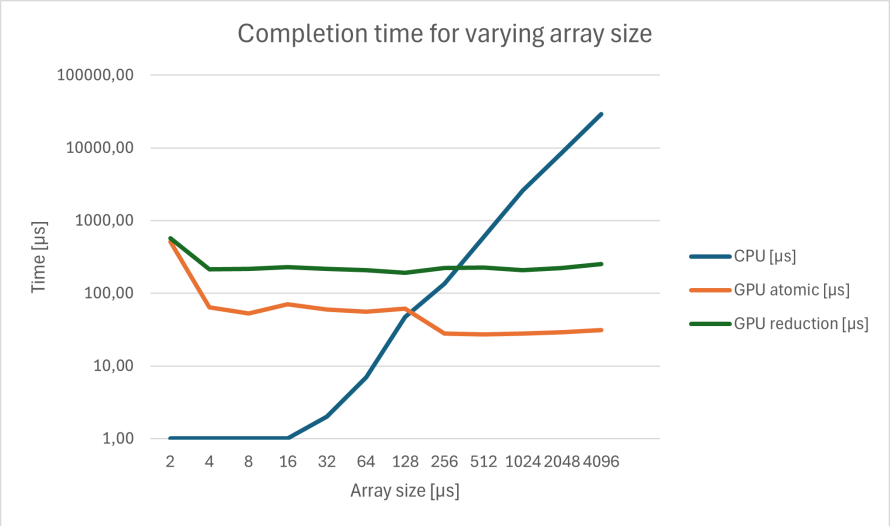


Figure 1: Completion time for varying array size

Bij kleine arrays presteert de CPU beter vanwege de overhead van GPU-opstart en geheugenoverdracht. Naarmate de array-grootte toeneemt, wordt de GPU efficiënter, omdat de parallelle verwerking beter schaalbaar is. Voor grote arrays groeit de CPU-tijd exponentieel. De Beide GPU implementaties hebben een eerder stabiele uitvoeringstijd ten opzichte van de array lengte. Over het algemeen heeft de atomic operation implementatie een lagere uitvoeringstijd dan de reduction implementatie.

4 Vragen

4.1 Wat gebeurt er als de invoer groter is dan 2048?

Vanaf 2048 elementen zijn er meerdere blokken nodig, aangezien de grootte van één blok beperkt is tot 1024 threads. Het probleem hierbij is dat __syncthreads() alleen binnen een blok werkt. Om dit op te lossen, laat elk blok afzonderlijk een deel van de lijst doorzoeken naar het maximum. Vervolgens zal de CPU de resultaten van de verschillende blokken verwerken om het globale maximum te bepalen.

4.2 Wat als er minder threads zijn dan data-elementen?

Als de optimale oplossing zich bevindt voorbij het punt dat gelijk is aan twee keer het aantal threads, zal deze maximale waarde niet worden gevonden.

4.3 Hoe zou je een 2D-array aanpakken?

Een mogelijke aanpak voor een 2D-array is om alle rijen afzonderlijk parallel te verwerken (de beste oplossingen worden naar de eerste index verplaatst door gebruik te maken van reduction). Vervolgens wordt in de eerste kolom de maximale index gezocht aan de hand van bijvoorbeeld reduction.

5 Appendix

5.1 CPU (Nested for-loop)

```cpp
#include <iostream>
#include <vector>
#include <limits.h>

int findMaxValue(const std::vector<int>& arr) {
    int maxVal = INT_MIN;
    for (size_t i = 0; i < arr.size(); ++i) {
        for (size_t j = i; j < arr.size(); ++j) {
            if (arr[j] > maxVal) {
                maxVal = arr[j];
            }
        }
    }
    return maxVal;
}

int main() {
    std::vector<int> arr = {1, 3, 5, 7, 9, 2, 4, 6, 8, 0};
    int maxVal = findMaxValue(arr);
    std::cout << "max value: " << maxVal << std::endl;
    return 0;
}
```

5.2 GPU kernel (atomic operation)

```cpp
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <cuda_runtime.h>

__global__ void findMaxValue(int *d_arr, int size, int *d_max) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    if (idx < size) {
        atomicMax(d_max, d_arr[idx]);
    }
}

int main() {
    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);
    printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);
    printf("Max Blocks per Grid: %d, %d, %d\n",
        prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);


    const int size = 50000;
    int h_arr[size];
    for (int i = 0; i < size; i++) {
        h_arr[i] = i;
    }

    int *d_arr;
    int h_max = INT_MIN;
    int *d_max;
```

```
31
32      cudaMalloc((void**)&d_arr, size * sizeof(int));
33      cudaMalloc((void**)&d_max, sizeof(int));
34
35      cudaMemcpy(d_arr, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);
36      cudaMemcpy(d_max, &h_max, sizeof(int), cudaMemcpyHostToDevice);
37
38      int threadsPerBlock = 1024;
39      int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
40      findMaxValue<<<blocksPerGrid, threadsPerBlock>>>(d_arr, size, d_max);
41
42      cudaMemcpy(&h_max, d_max, sizeof(int), cudaMemcpyDeviceToHost);
43
44      printf("Maximum value in the array: %d\n", h_max);
45
46
47      cudaFree(d_arr);
48      cudaFree(d_max);
49      return 0;
50  }
```

5.3 GPU kernel (reduction)

```
1   #include <stdio.h>
2   #include <chrono>
3   #include <iostream>
4   #include <cassert>
5   #include <cmath>
6   #include <cuda_runtime.h>
7   #include <thread>
8   #include <list>
9
10  __global__ void findMaxValue(int *d_arr, int size, int d_depth, int d_threads) {
11      int idx = threadIdx.x;
12      int offset = blockIdx.x * blockDim.x * 2;
13      //printf("id: %d, block: %d, offset: %d\n", idx, blockIdx.x, offset);
14
15      for(int i = 0; i < d_depth; i++) {
16          int exp = std::pow(2,i);
17          if(idx < (d_threads)/exp) {
18              int index1 = idx*2*exp;
19              int index2 = index1 + exp;
20              //printf("index1: %d, index2: %d, i: %d\n", index1, index2, i);
21
22              if(d_arr[offset+index1] < d_arr[offset+index2]) {
23                  d_arr[offset+index1] = d_arr[offset+index2];
24              }
25          }
26          // Alleen in de blokken
27          __syncthreads();
28      }
29  }
30
31  int main() {
32      cudaDeviceProp prop;
33      cudaGetDeviceProperties(&prop, 0);
34      printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);
35      printf("Max Blocks per Grid: %d, %d, %d\n",
36          prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);
37
```

```
38
39      const int size = 4096;
40      //const int size = 2048;
41      int h_arr[size];
42      for (int i = 0; i < size; i++) {
43          h_arr[i] = i;
44      }
45
46      int *d_arr;  // Device array
47      int d_depth;
48      int d_threads;
49
50
51      cudaMalloc((void**)&d_arr, size * sizeof(int));
52      cudaMemcpy(d_arr, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);
53
54
55      int threads = size/2;
56      d_threads = threads;
57      int threadsPerBlock = std::min(1024, d_threads);
58      int blocksPerGrid = threads / threadsPerBlock;
59
60      printf("threadsPerBlock: %d, blocksPerGrid: %d\n", threadsPerBlock,
            blocksPerGrid);
61      assert(threadsPerBlock <= prop.maxThreadsPerBlock);
62
63      int h_depth = std::log2(threadsPerBlock);
64
65      d_depth = h_depth;
66
67      findMaxValue<<<blocksPerGrid, threadsPerBlock>>>(d_arr, size, d_depth,
            d_threads);
68
69
70      cudaMemcpy(&h_arr, d_arr, size * sizeof(int), cudaMemcpyDeviceToHost);
71      cudaDeviceSynchronize();
72
73      int max = INT_MIN;
74
75      for(int i = 0; i < blocksPerGrid*2; i++) {
76          int value = h_arr[i*threadsPerBlock];
77          if(value > max) {
78              max = value;
79          }
80      }
81
82      printf("Maximum value in the array: %d\n", max);
83
84      cudaFree(d_arr);
85      return 0;
86  }
```